specific structure of APT repositories (for instance they know when individual files are obsolete or not, and thus adjust the time during which they are kept).

apt-cacher and *apt-cacher-ng* work like usual proxy cache servers. APT's `sources.list` is left unchanged, but APT is configured to use them as proxy for outgoing requests.

approx, on the other hand, acts like an HTTP server that "mirrors" any number of remote repositories in its top-level URLs. The mapping between those top-level directories and the remote URLs of the repositories is stored in `/etc/approx/approx.conf`:

```
# <name> <repository-base-url>
debian   https://deb.debian.org/debian
security http://security.debian.org
```

approx runs by default on port 9999 via a systemd socket and requires the users to adjust their `sources.list` file to point to the approx server:

```
# Sample sources.list pointing to a local approx server
deb http://localhost:9999/security buster/updates main contrib non-free
deb http://localhost:9999/debian buster main contrib non-free
```

6.2. `aptitude`, `apt-get`, and `apt` Commands

APT is a vast project, whose original plans included a graphical interface. It is based on a library which contains the core application, and `apt-get` is the first front end — command-line based — which was developed within the project. `apt` is a second command-line based front end provided by APT which overcomes some design mistakes of `apt-get`.

Both tools are built on top of the same library and are thus very close, but the default behavior of `apt` has been improved for interactive use and to actually do what most users expect. The APT developers reserve the right to change the public interface of this tool to further improve it. On the opposite, the public interface of `apt-get` is well defined and will not change in any backwards incompatible way. It is thus the tool that you want to use when you need to script package installation requests.

Numerous other graphical interfaces then appeared as external projects: `synaptic`, `aptitude` (which includes both a text mode interface and a graphical one — even if not complete yet), `wajig`, etc. The most recommended interface, `apt`, is the one that we will use in the examples given in this section. Note, however, that `apt-get` and `aptitude` have a very similar command line syntax. When there are major differences between these three commands, these will be detailed.

6.2.1. Initialization

For any work with APT, the list of available packages needs to be updated; this can be done simply through `apt update`. Depending on the speed of your connection and configuration, the op-

eration can take a while, since it involves downloading a certain number of (usually compressed) files (`Packages`, `Sources`, `Translation-`*`language-code`*), which have gradually become bigger and bigger as Debian has developed (at least 10 MB of data for the main section). Of course, installing from a CD-ROM/DVD set does not require any downloading — in this case, the operation is very fast.

TIP

Incremental updates

The aim of the `apt update` command is to download for each package source the corresponding `Packages` (or `Sources`) file. However, even after a `xz` compression, these files can remain rather large (the `Packages.xz` for the *main* section of *Buster* takes more than 7 MB). If you wish to update regularly, these downloads can take up a lot of time.

To speed up the process APT can download "diff" files containing the changes since the previous update, as opposed to the entire file. To achieve this, official Debian mirrors distribute different files which list the differences between one version of the `Packages` file and the following version. They are generated at each update of the archives and a history of one week is kept. Each of these "diff" files only takes a few dozen kilobytes for *Unstable*, so that the amount of data downloaded by a weekly `apt update` is often divided by 10. For *Stable* and *Testing*, which change less, the gain is even more noticeable.

However, it can sometimes be of interest to force the download of the entire `Packages` file, especially when the last upgrade is very old and when the mechanism of incremental differences would not contribute much. This can also be interesting when network access is very fast but when the processor of the machine to upgrade is rather slow, since the time saved on the download is more than lost when the computer calculates the new versions of these files (starting with the older versions and applying the downloaded differences). To do that, you can use the APT configuration parameter `Acquire::PDiffs` and set it to `false`.

```
$ sudo apt -o "Acquire::PDiffs=false" update
```

The `Acquire::*` options also control other aspects of the download, and even the download methods. `Acquire::Languages` can limit or disable the download of `Translation-`*`language-code`* files and save even more time. For a complete reference see `apt.conf(5)`.

6.2.2. Installing and Removing

With APT, packages can be added or removed from the system, respectively with `apt install` *`package`* and `apt remove` *`package`*. In both cases, APT will automatically install the necessary dependencies or delete the packages which depend on the package that is being removed. The `apt purge` *`package`* command involves a complete uninstallation by deleting the configuration files as well.

TIP

Installing the same selection of packages several times

It can be useful to systematically install the same list of packages on several computers. This can be done quite easily.

First, retrieve the list of packages installed on the computer which will serve as the "model" to copy.

```
$ dpkg --get-selections >pkg-list
```

The `pkg-list` file then contains the list of installed packages. Next, transfer the `pkg-list` file onto the computers you want to update and use the following commands:

```
## Update dpkg's database of known packages
# avail=`mktemp`
# apt-cache dumpavail > "$avail"
# dpkg --merge-avail "$avail"
# rm -f "$avail"
## Update dpkg's selections
# dpkg --set-selections < pkg-list
## Ask apt-get to install the selected packages
# apt-get dselect-upgrade
```

The first commands record the list of available packages in the dpkg database. Then `dpkg --set-selections` restores the selection of packages that you wish to install, and the `apt-get` invocation executes the required operations! `aptitude` does not have this command.

TIP

Removing and installing at the same time

It is possible to ask `apt` (or `apt-get`, or `aptitude`) to install certain packages and remove others on the same command line by adding a suffix. With an `apt install` command, add "`-`" to the names of the packages you wish to remove. With an `apt remove` command, add "+" to the names of the packages you wish to install.

The next example shows two different ways to install *package1* and to remove *package2*.

```
# apt install package1 package2-
```

```
# apt remove package1+ package2
```

This can also be used to exclude packages which would otherwise be installed, for example, due to an automatic installation of `Recommends`. In general, the dependency solver will use that information as a hint to look for alternative solutions.

TIP

`apt --reinstall` and `aptitude reinstall`

The system can sometimes be damaged after the removal or modification of files in a package. The easiest way to retrieve these files is to reinstall the affected package. Unfortunately, the packaging system finds that the latter is already installed and politely refuses to reinstall it; to avoid this, use the `--reinstall` option of the `apt` and `apt-get` commands. The following command reinstalls *postfix* even if it is already present:

```
# apt --reinstall install postfix
```

The `aptitude` command line is slightly different, but achieves the same result with `aptitude reinstall postfix`.

The problem does not arise with `dpkg`, but the administrator rarely uses it directly.

Be careful! Using `apt --reinstall` to restore packages modified during an attack will certainly not recover the system as it was. section 14.7, "Dealing with a Compromised Machine" page 440 details the necessary steps to take with a compromised system.

These commands will not restore the configuration files. But as you have learned in section 5.2.3, "Checksums, List of Configuration Files" page 89 (see also sidebar "Force dpkg to ask configuration file questions" page 89), you can use the following command to be asked to install the unmodified version and even restore any deleted configuration file as well.

```
# apt --reinstall -o Dpkg::Options::="--force-confask,
    ➥ confmiss" install package
```

Some packages don't ship the configuration file found in `/etc` with the package. Instead they create it during installation by either copying a skeleton or writing it by a script. The file `/etc/inputrc`, for example, is a copy of `/usr/share/readline/inputrc`. In such cases the commands shown above won't work.

If the file `sources.list` mentions several distributions, it is possible to give the version of the package to install. A specific version number can be requested with `apt install package=version`, but indicating its distribution of origin (*Stable*, *Testing* or *Unstable*) — with `apt install package/distribution` — is usually preferred. With this command, it is possible to go back to an older version of a package (if, for instance, you know that it works well), provided that it is still available in one of the sources referenced by the `sources.list` file. Otherwise the snapshot.debian.org archive can come to the rescue (see sidebar "Old package versions: `snapshot.debian.org`" page 115).

Example 6.4 *Installation of the* Unstable *version of* spamassassin

```
# apt install spamassassin/unstable
```

If the package to install has been made available to you under the form of a simple `.deb` file without any associated package repository, it is still possible to use APT to install it together with its dependencies (provided that the dependencies are available in the configured repositories) with a simple command: `apt install ./path-to-the-package.deb`. The leading ./ is important to make it clear that we are referring to a filename and not to the name of a package available in one of the repositories.

GOING FURTHER

The cache of `.deb` files

APT keeps a copy of each downloaded `.deb` file in the directory `/var/cache/apt/archives/`. In case of frequent updates, this directory can quickly take a lot of disk space with several versions of each package; you should regularly sort through them. Two commands can be used: `apt-get clean` entirely empties the directory; `apt-get autoclean` only removes packages which can no longer be downloaded (because they have disappeared from the Debian mirror) and are therefore clearly useless (the configuration parameter `APT::Clean-Installed` can prevent the removal of `.deb` files that are currently installed).

6.2.3. System Upgrade

Regular upgrades are recommended, because they include the latest security updates. To upgrade, use `apt upgrade`, `apt-get upgrade` or `aptitude safe-upgrade` (of course after `apt update`). This command looks for installed packages which can be upgraded without removing any packages. In other words, the goal is to ensure the least intrusive upgrade possible. `apt-get` is slightly more demanding than `aptitude` or `apt` because it will refuse to install packages which were not installed beforehand.

`apt` will generally select the most recent version number (except for packages from *Experimental* and *stable-backports*, which are ignored by default whatever their version number). If you specified *Testing* or *Unstable* in your `sources.list`, `apt upgrade` will switch most of your *Stable* system to *Testing* or *Unstable*, which might not be what you intended.

To tell `apt` to use a specific distribution when searching for upgraded packages, you need to use the -t or --target-release option, followed by the name of the distribution you want (for example, `apt -t stable upgrade`). To avoid specifying this option every time you use `apt`, you can add APT::Default-Release "stable"; in the file `/etc/apt/apt.conf.d/local`.

For more important upgrades, such as the change from one major Debian version to the next, you need to use `apt full-upgrade`. With this instruction, `apt` will complete the upgrade even if it has to remove some obsolete packages or install new dependencies. This is also the command used by users who work daily with the Debian *Unstable* release and follow its evolution day by day. It is so simple that it hardly needs explanation: APT's reputation is based on this great functionality.

Unlike `apt` and `aptitude`, `apt-get` doesn't know the `full-upgrade` command. Instead, you should use `apt-get dist-upgrade` ("distribution upgrade"), the historical and well-known command that `apt` and `aptitude` also accept for the convenience of users who got used to it.

The results of these operations are logged into `/var/log/apt/history.log` and `/var/log/apt/term.log`, whereas `dpkg` keeps its log in a file called `/var/log/dpkg.log`.

6.2.4. Configuration Options

Besides the configuration elements already mentioned, it is possible to configure certain aspects of APT by adding directives in a file of the `/etc/apt/apt.conf.d/` directory or `/etc/apt/apt.conf` itself. Remember, for instance, that it is possible for APT to tell `dpkg` to ignore file conflict errors by specifying DPkg::options { "--force-overwrite"; }.

If the Web can only be accessed through a proxy, add a line like Acquire::http::proxy "http://*yourproxy*:3128". For an FTP proxy, write Acquire::ftp::proxy "ftp://*yourproxy*". To discover more configuration options, read the `apt.conf(5)` manual page with the `man apt.conf` command (for details on manual pages, see section 7.1.1, "Manual Pages" page 148).

BACK TO BASICS

Directories ending in `.d`

Directories with a `.d` suffix are used more and more often. Each directory represents a configuration file which is split over multiple files. In this sense, all of the

files in `/etc/apt/apt.conf.d/` are instructions for the configuration of APT. APT includes them in alphabetical order, so that the last ones can modify a configuration element defined in one of the first ones.

This structure brings some flexibility to the machine administrator and to the package maintainers. Indeed, the administrator can easily modify the configuration of the software by adding a ready-made file in the directory in question without having to change an existing file. Package maintainers use the same approach when they need to adapt the configuration of another software to ensure that it perfectly co-exists with theirs. The Debian policy explicitly forbids modifying configuration files of other packages — only users are allowed to do this. Remember that during a package upgrade, the user gets to choose the version of the configuration file that should be kept when a modification has been detected. Any external modification of the file would trigger that request, which would disturb the administrator, who is sure not to have changed anything.

Without a `.d` directory, it is impossible for an external package to change the settings of a program without modifying its configuration file. Instead it must invite the user to do it themselves and lists the operations to be done in the file `/usr/share/doc/`*`package`*`/README.Debian`.

Depending on the application, the `.d` directory is used directly or managed by an external script which will concatenate all the files to create the configuration file itself. It is important to execute the script after any change in that directory so that the most recent modifications are taken into account. In the same way, it is important not to work directly in the configuration file created automatically, since everything would be lost at the next execution of the script. The chosen method (`.d` directory used directly or a file generated from that directory) is usually dictated by implementation constraints, but in both cases the gains in terms of configuration flexibility more than make up for the small complications that they entail. The Exim 4 mail server is an example of the generated file method: it can be configured through several files (`/etc/exim4/conf.d/*`) which are concatenated into `/var/lib/exim4/config.autogenerated` by the `update-exim4.conf` command.

6.2.5. Managing Package Priorities

One of the most important aspects in the configuration of APT is the management of the priorities associated with each package source. For instance, you might want to extend one distribution with one or two newer packages from *Testing*, *Unstable* or *Experimental*. It is possible to assign a priority to each available package (the same package can have several priorities depending on its version or the distribution providing it). These priorities will influence APT's behavior: for each package, it will always select the version with the highest priority (except if this version is older than the installed one and if its priority is less than 1000).

APT defines several default priorities. Each installed package version has a priority of 100. A non-installed version has a priority of 500 by default, but it can jump to 990 if it is part of the target release (defined with the -t command-line option or the APT::Default-Release configuration directive).

You can modify the priorities by adding entries in a file in `/etc/apt/preferences.d/` or the `/etc/apt/preferences` file with the names of the affected packages, their version, their origin and their new priority.

APT will never install an older version of a package (that is, a package whose version number is lower than the one of the currently installed package) except if its priority is higher than 1000 (or it is explicitely requested by the user, see section 6.2.2, "Installing and Removing" page 117). APT will always install the highest priority package which follows this constraint. If two packages have the same priority, APT installs the newest one (whose version number is the highest). If two packages of same version have the same priority but differ in their content, APT installs the version that is not installed (this rule has been created to cover the case of a package update without the increment of the revision number, which is usually required).

In more concrete terms, a package whose priority is

< 0 will never be installed,

1..99 will only be installed if no other version of the package is already installed,

100..499 will only be installed if there is no other newer version installed or available in another distribution,

500....989 will only be installed if there is no newer version installed or available in the target distribution,

990..1000 will be installed except if the installed version is newer,

> 1000 will always be installed, even if it forces APT to downgrade to an older version.

When APT checks `/etc/apt/preferences` and `/etc/apt/preferences.d/`, it first takes into account the most specific entries (often those specifying the concerned package), then the more generic ones (including, for example, all the packages of a distribution). If several generic entries exist, the first match is used. The available selection criteria include the package's name and the source providing it. Every package source is identified by the information contained in a `Release` file that APT downloads together with the `Packages` files. It specifies the origin (usually "Debian" for the packages of official mirrors, but it can also be a person's or an organization's name for third-party repositories). It also gives the name of the distribution (usually *Stable*, *Testing*, *Unstable* or *Experimental* for the standard distributions provided by Debian) together with its version (for example, 10 for Debian *Buster*). Let's have a look at its syntax through some realistic case studies of this mechanism.

SPECIFIC CASE

Priority of *experimental*

If you listed *Experimental* in your `sources.list` file, the corresponding packages will almost never be installed because their default APT priority is 1. This is of course a specific case, designed to keep users from installing *Experimental* packages by mistake. The packages can only be installed by typing `aptitude install` *package*`/experimental` — users typing this command can only be aware of the risks that they take. It is still possible (though *not* recommended) to treat packages of *Experimental* like those of other distributions by giving them a priority of 500. This is done with a specific entry in `/etc/apt/preferences`:

```
Package: *
Pin: release a=experimental
Pin-Priority: 500
```

Let's suppose that you only want to use packages from the stable version of Debian. Those provided in other versions should not be installed except if explicitly requested. You could write the following entries in the `/etc/apt/preferences` file:

```
Package: *
Pin: release a=stable
Pin-Priority: 900

Package: *
Pin: release o=Debian
Pin-Priority: -10
```

a=stable defines the name of the selected distribution. o=Debian limits the scope to packages whose origin is "Debian".

Let's now assume that you have a server with several local programs depending on the version 5.24 of Perl and that you want to ensure that upgrades will not install another version of it. You could use this entry:

```
Package: perl
Pin: version 5.24*
Pin-Priority: 1001
```

To gain a better understanding of the mechanisms of priority and distribution or repository properties to pin do not hesitate to execute `apt-cache policy` to display the default priority associated with each package source, or `apt-cache policy` *`package`* to display the default priority for each available version and source of a package as explained in "`apt-cache policy`" page 127.

The reference documentation for the files `/etc/apt/preferences` and `/etc/apt/preferences.d/` is available in the manual page `apt_preferences(5)`, which you can display with `man apt_preferences`.

TIP

Comments in `/etc/apt/preferences`

There is no official syntax to put comments in the `/etc/apt/preferences` file, but some textual descriptions can be provided by putting one or more "`Explanation`" fields at the start of each entry:

```
Explanation: The package xserver-xorg-video-intel provided
Explanation: in experimental can be used safely
Package: xserver-xorg-video-intel
Pin: release a=experimental
Pin-Priority: 500
```

`apt` being such a marvelous tool, it is tempting to pick packages coming from other distributions. For example, after having installed a *Stable* system, you might want to try out a software package available in *Testing* or *Unstable* without diverging too much from the system's initial state.

Even if you will occasionally encounter problems while mixing packages from different distributions, `apt` manages such coexistence very well and limits risks very effectively. The best way to proceed is to list all distributions used in `/etc/apt/sources.list` (some people always put the three distributions, but remember that *Unstable* is reserved for experienced users) and to define your reference distribution with the APT::Default-Release parameter (see section 6.2.3, "System Upgrade" page 120).

Let's suppose that *Stable* is your reference distribution but that *Testing* and *Unstable* are also listed in your `sources.list` file. In this case, you can use `apt install package/testing` to install a package from *Testing*. If the installation fails due to some unsatisfiable dependencies, let it solve those dependencies within *Testing* by adding the -t testing parameter. The same obviously applies to *Unstable*.

In this situation, upgrades (`upgrade` and `full-upgrade`) are done within *Stable* except for packages already upgraded to another distribution: those will follow updates available in the other distributions. We will explain this behavior with the help of the default priorities set by APT below. Do not hesitate to use `apt-cache policy` (see sidebar "`apt-cache policy`" page 127) to verify the given priorities.

Everything centers around the fact that APT only considers packages of higher or equal version than the installed one (assuming that `/etc/apt/preferences` has not been used to force priorities higher than 1000 for some packages).

Let's assume that you have installed version 1 of a first package from *Stable* and that version 2 and 3 are available respectively in *Testing* and *Unstable*. The installed version has a priority of 100 but the version available in *Stable* (the very same) has a priority of 990 (because it is part of the target release). Packages in *Testing* and *Unstable* have a priority of 500 (the default priority of a non-installed version). The winner is thus version 1 with a priority of 990. The package "stays in *Stable*".

Let's take the example of another package whose version 2 has been installed from *Testing*. Version 1 is available in *Stable* and version 3 in *Unstable*. Version 1 (of priority 990 — thus lower than 1000) is discarded because it is lower than the installed version. This only leaves version 2 and 3, both of priority 500. Faced with this alternative, APT selects the newest version, the one from *Unstable*. If you don't want a package installed from *Testing* to migrate to *Unstable*, you have to assign a priority lower than 500 (490 for example) to packages coming from *Unstable*. You can modify `/etc/apt/preferences` to this effect:

```
Package: *
Pin: release a=unstable
Pin-Priority: 490
```

6.2.7. Tracking Automatically Installed Packages

One of the essential functionalities of `apt` is the tracking of packages installed only through dependencies. These packages are called "automatic", and often include libraries.

With this information, when packages are removed, the package managers can compute a list of automatic packages that are no longer needed (because there is no "manually installed" packages depending on them). `apt-get autoremove` or `apt autoremove` will get rid of those packages. `aptitude` does not have this command because it removes them automatically as soon as they are identified. In all cases, the tools display a clear message listing the affected packages.

It is a good habit to mark as automatic any package that you don't need directly so that they are automatically removed when they aren't necessary anymore. `apt-mark auto` *`package`* will mark the given package as automatic whereas `apt-mark manual` *`package`* does the opposite. `aptitude markauto` and `aptitude unmarkauto` work in the same way although they offer more features for marking many packages at once (see section 6.5.1, "aptitude" page 128). The console-based interactive interface of `aptitude` also makes it easy to review the "automatic flag" on many packages.

People might want to know why an automatically installed package is present on the system. To get this information from the command line, you can use `aptitude why` *`package`* (`apt` and `apt-get` have no similar feature):

```
$ aptitude why python-debian
i   aptitude         Suggests apt-xapian-index
p   apt-xapian-index Depends  python-debian (>= 0.1.14)
```

ALTERNATIVE

`deborphan` and `debfoster`

In days where `apt`, `apt-get` and `aptitude` were not able to track automatic packages, there were two utilities producing lists of unnecessary packages: `deborphan` and `debfoster`. Both can still be useful.

`deborphan` scans the `libs` and `oldlibs` sections (in the absence of supplementary instructions) by default looking for the packages that are currently installed and that no other package depends on. The resulting list can then serve as a basis to remove unneeded packages.

`debfoster` has a more elaborate approach, very similar to APT's one: it maintains a list of packages that have been explicitly installed, and remembers what packages are really required between each invocation. If new packages appear on the system and if `debfoster` doesn't know them as required packages, they will be shown on the screen together with a list of their dependencies. The program then offers a choice: remove the package (possibly together with those that depend on it), mark it as explicitly required, or ignore it temporarily.

6.3. The apt-cache Command

The apt-cache command can display much of the information stored in APT's internal database. This information is a sort of cache since it is gathered from the different sources listed in the sources.list file. This happens during the apt update operation.

VOCABULARY

Cache

A cache is a temporary storage system used to speed up frequent data access when the usual access method is expensive (performance-wise). This concept can be applied in numerous situations and at different scales, from the core of microprocessors up to high-end storage systems.

In the case of APT, the reference Packages files are those located on Debian mirrors. That said, it would be very ineffective to go through the network for every search that we might want to do in the database of available packages. That is why APT stores a copy of those files (in /var/lib/apt/lists/) and searches are done within those local files. Similarly, /var/cache/apt/archives/ contains a cache of already downloaded packages to avoid downloading them again if you need to reinstall them after a removal.

On the other hand, it is mandatory to run apt update regularly to update the cache. Otherwise your package search results will always miss the latest updates distributed by the Debian mirrors.

The apt-cache command can do keyword-based package searches with apt-cache search *keyword*. It can also display the headers of the package's available versions with apt-cache show *package*. This command provides the package's description, its dependencies, the name of its maintainer, etc. Note that apt search, apt show, aptitude search, aptitude show work in the same way.

ALTERNATIVE

axi-cache

apt-cache search is a very rudimentary tool, basically implementing grep on package's descriptions. It often returns too many results or none at all when you include too many keywords.

axi-cache search *term*, on the other hand, provides better results, sorted by relevancy. It uses the *Xapian* search engine and is part of the *apt-xapian-index* package which indexes all package information (and more, like the .desktop files from all Debian packages). It knows about tags (see sidebar "The Tag field" page 86) and returns results in a matter of milliseconds.

```
$ axi-cache search package use::searching

100 results found.
Results 1-20:
100% packagesearch - GUI for searching packages and viewing
    ➥  package information
99% apt-utils - package management related utility programs
98% whohas - query multiple distributions' package archives
98% dpkg-awk - Gawk script to parse /var/lib/dpkg/{status,
    ➥ available} and Packages
97% apt-file - search for files within Debian packages (
    ➥ command-line interface)
```

```
[..]
90% wajig - unified package management front-end for Debian
More terms: debtags debian paket dpkg search pakete tools
More tags: role::program interface::commandline works-with
    ➥ ::software:package suite::debian admin::package-
    ➥ management scope::utility network::client
‘axi-cache more’ will give more results
```

Some features are more rarely used. For instance, `apt-cache policy` displays the priorities of package sources as well as those of individual packages. Another example is `apt-cache dumpavail` which displays the headers of all available versions of all packages. `apt-cache pkgnames` displays the list of all the packages which appear at least once in the cache.

TIP

apt-cache policy

The `apt-cache policy` command displays the pinning priorities and distribution properties of each package source as explained in section 6.2.5, “Managing Package Priorities” page 121. It can also show the pinning priorities for all available versions and sources of a package. For the `sources.list` example used in Example 6.2, “`/etc/apt/sources.list` file for users of Debian Stable” page 110 and `APT::Default-Release` set to `"buster"`, the output will look like this:

```
$ apt-cache policy
Package files:
 100 /var/lib/dpkg/status
     release a=now
 100 https://deb.debian.org/debian buster-backports/contrib amd64 Packages
     release o=Debian Backports,a=buster-backports,n=buster-backports,l=Debian Backports,c=contrib,
         ➥ b=amd64
     origin deb.debian.org
 100 https://deb.debian.org/debian buster-backports/main amd64 Packages
     release o=Debian Backports,a=buster-backports,n=buster-backports,l=Debian Backports,c=main,b=
         ➥ amd64
     origin deb.debian.org
 990 https://deb.debian.org/debian buster/non-free amd64 Packages
     release v=10.0,o=Debian,a=stable,n=buster,l=Debian,c=non-free,b=amd64
     origin deb.debian.org
 990 https://deb.debian.org/debian buster/contrib amd64 Packages
     release v=10.0,o=Debian,a=stable,n=buster,l=Debian,c=contrib,b=amd64
     origin deb.debian.org
 990 https://deb.debian.org/debian buster/main amd64 Packages
     release v=10.0,o=Debian,a=stable,n=buster,l=Debian,c=main,b=amd64
     origin deb.debian.org
 990 http://security.debian.org buster/updates/main amd64 Packages
     release v=10,o=Debian,a=stable,n=buster,l=Debian-Security,c=main,b=amd64
     origin security.debian.org
```

`apt-cache policy` can also show the pinning priorities for all available versions and sources of a given package.

```
$ apt-cache policy iptables
iptables:
  Installed: 1.8.2-4
  Candidate: 1.8.2-4
  Version table:
     1.8.3-2~bpo10+1 100
        100 https://deb.debian.org/debian buster-backports/main amd64 Packages
 *** 1.8.2-4 990
        990 https://deb.debian.org/debian buster/main amd64 Packages
        100 /var/lib/dpkg/status
```

Although there is a newer version of *iptables* in the `buster-backports` repository, APT will not install it automatically based on the priority. One would have to use `apt install iptables/buster-backports` or add a higher pinning priority to `/etc/apt/preferences.d/iptables`:

```
Package: iptables
```

```
Pin: release o=Debian Backports, a=buster-backports
Pin-Priority: 1001
```

6.4. The `apt-file` Command

Sometimes we refer to a file or a command and you might wonder, in which package it will be found. Fortunately the Debian repositories not only contain information about all the binary packages provided, but also all the files shipped with them. This information is stored in files named `Contents-`*`arch`*`.gz` and `Contents-udeb-`*`arch`*`.gz`. This information is not automatically downloaded by APT. Instead it needs the `apt-file update` command (from the similar named package) to retrieve the contents of all package sources mentioned in `/etc/apt/sources.list`. To update the database on a weekly base, the following entry can be added to `/etc/crontab` if convenient.

```
@weekly root test -x /usr/bin/apt-file && /usr/bin/apt-file update >> /dev/null 2>&1
```

After the database has been updated, the command `apt-file search` *`pattern`* will list all packages, which contain a filename or path containing the pattern.

```
$ apt-file search bin/axi-cache
apt-xapian-index: /usr/bin/axi-cache
```

The command `apt-file list` *`package`* will list all files shipped with the package instead.

TIP

Listing a package contents and finding a file's package

Similar to `apt-file list` the command `dpkg -L` *`package`* lists all files, but only for an installed package. To find the package, a local file belongs to, use `dpkg -S` *`file`* (see section 5.4.3, "Querying dpkg's Database and Inspecting `.deb` Files" page 96). To list all local files not belonging to any installed package, you might want to take a look at the *cruft* or the *cruft-ng* package.

6.5. Frontends: `aptitude`, `synaptic`

APT is a C++ program whose code mainly resides in the libapt-pkg shared library. Using a shared library facilitates the creation of user interfaces (front-ends), since the code contained in the library can easily be reused. Historically, `apt-get` was only designed as a test front-end for libapt-pkg but its success tends to obscure this fact.

6.5.1. `aptitude`

`aptitude` is an interactive program that can be used in semi-graphical mode on the console. You can browse the list of installed and available packages, look up all the available information, and select packages to install or remove. The program is designed specifically to be used

by administrators, so that its default behaviors are designed to be much more intelligent than `apt-get`'s, and its interface much easier to understand.

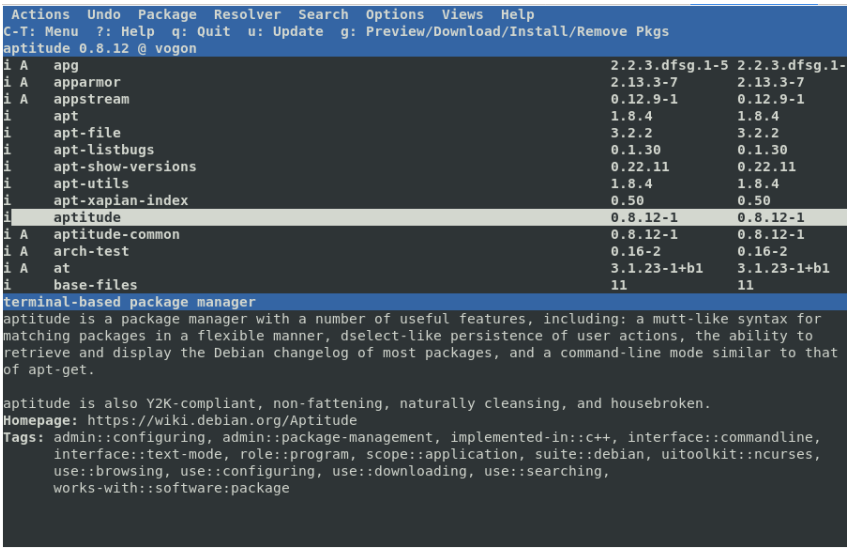


Figure 6.1 *The `aptitude` package manager*

When it starts, `aptitude` shows a list of packages sorted by state (installed, non-installed, or installed but not available on the mirrors — other sections display tasks, virtual packages, and new packages that appeared recently on mirrors). To facilitate thematic browsing, other views are available. In all cases, `aptitude` displays a list combining categories and packages on the screen. Categories are organized through a tree structure, whose branches can respectively be unfolded or closed with the Enter, [and] keys. + should be used to mark a package for installation, - to mark it for removal and _ to purge it (note that these keys can also be used for categories, in which case the corresponding actions will be applied to all the packages of the category). u updates the lists of available packages and Shift+u prepares a global system upgrade. g switches to a summary view of the requested changes (and typing g again will apply the changes), and q quits the current view. If you are in the initial view, this will effectively close `aptitude`.

DOCUMENTATION

`aptitude`

This section does not cover the finer details of using `aptitude`. It rather focuses on giving you a survival kit to use it. But it is well documented and we advise you to use its complete manual available in the *aptitude-doc-en* package (see `/usr/share/doc/aptitude/html/en/index.html`) or at https://www.debian.org/doc/manuals/aptitude/).

To search for a package, you can type / followed by a search pattern. This pattern matches the name of the package, but can also be applied to the description (if preceded by ~d), to the section (with ~s) or to other characteristics detailed in the documentation. The same patterns can filter the list of displayed packages: type the l key (as in *limit*) and enter the pattern.

Managing the "automatic flag" of Debian packages (see section 6.2.7, "Tracking Automatically Installed Packages" page 125) is a breeze with `aptitude`. It is possible to browse the list of installed packages and mark packages as automatic with Shift+m or to remove the mark with the m key. "Automatic packages" are displayed with an "A" in the list of packages. This feature also offers a simple way to visualize the packages in use on a machine, without all the libraries and dependencies that you don't really care about. The related pattern that can be used with l (to activate the filter mode) is ~i!~M. It specifies that you only want to see installed packages (~i) not marked as automatic (!~M).

TOOL

Using aptitude on the command-line interface

Most of `aptitude`'s features are accessible via the interactive interface as well as via command-lines. These command-lines will seem familiar to regular users of `apt-get` and `apt-cache`.

The advanced features of `aptitude` are also available on the command-line. You can use the same package search patterns as in the interactive version. For example, if you want to cleanup the list of "manually installed" packages, and if you know that none of the locally installed programs require any particular libraries or Perl modules, you can mark the corresponding packages as automatic with a single command:

```
# aptitude markauto '~slibs|~sperl'
```

Here, you can clearly see the power of the search pattern system of `aptitude`, which enables the instant selection of all the packages in the `libs` and `perl` sections.

Beware, if some packages are marked as automatic and if no other package depends on them, they will be removed immediately (after a confirmation request).

Managing Recommendations, Suggestions and Tasks

Another interesting feature of `aptitude` is the fact that it respects recommendations between packages while still giving users the choice not to install them on a case by case basis. For example, the *gnome* package recommends *transmission-gtk* (among others). When you select the former for installation, the latter will also be selected (and marked as automatic if not already installed on the system). Typing g will make it obvious: *transmission-gtk* appears on the summary screen of pending actions in the list of packages installed automatically to satisfy dependencies. However, you can decide not to install it by deselecting it before confirming the operations.

Note that this recommendation tracking feature does not apply to upgrades. For instance, if a new version of *gnome* recommends a package that it did not recommend formerly, the package won't be marked for installation. However, it will be listed on the upgrade screen so that the administrator can still select it for installation.

Suggestions between packages are also taken into account, but in a manner adapted to their specific status. For example, since *gnome* suggests *empathy*, the latter will be displayed on the summary screen of pending actions (in the section of packages suggested by other packages). This way, it is visible and the administrator can decide whether to take the suggestion into

account or not. Since it is only a suggestion and not a dependency or a recommendation, the package will not be selected automatically — its selection requires a manual intervention from the user (thus, the package will not be marked as automatic).

In the same spirit, remember that `aptitude` makes intelligent use of the concept of task. Since tasks are displayed as categories in the screens of packages lists, you can either select a full task for installation or removal, or browse the list of packages included in the task to select a smaller subset.

Better Solver Algorithms

To conclude this section, let's note that `aptitude` has more elaborate algorithms compared to `apt-get` when it comes to resolving difficult situations. When a set of actions is requested and when these combined actions would lead to an incoherent system, `aptitude` evaluates several possible scenarios and presents them in order of decreasing relevance. However, these algorithms are not failproof. Fortunately there is always the possibility to manually select the actions to perform. When the currently selected actions lead to contradictions, the upper part of the screen indicates a number of "broken" packages (and you can directly navigate to those packages by pressing b). It is then possible to manually build a solution for the problems found. In particular, you can get access to the different available versions by simply selecting the package with Enter. If the selection of one of these versions solves the problem, you should not hesitate to use the function. When the number of broken packages gets down to zero, you can safely go to the summary screen of pending actions for a last check before you apply them.

NOTE

`aptitude`'s log

Like `dpkg`, `aptitude` keeps a trace of executed actions in its logfile (`/var/log/aptitude`). However, since both commands work at a very different level, you cannot find the same information in their respective logfiles. While `dpkg` logs all the operations executed on individual packages step by step, `aptitude` gives a broader view of high-level operations like a system-wide upgrade.

Beware, this logfile only contains a summary of operations performed by `aptitude`. If other front-ends (or even `dpkg` itself) are occasionally used, then `aptitude`'s log will only contain a partial view of the operations, so you can't rely on it to build a trustworthy history of the system.

6.5.2. `synaptic`

`synaptic` is a graphical package manager for Debian which features a clean and efficient graphical interface based on GTK+/GNOME. Its many ready-to-use filters give fast access to newly available packages, installed packages, upgradable packages, obsolete packages and so on. If you browse through these lists, you can select the operations to be done on the packages (install, upgrade, remove, purge); these operations are not performed immediately, but put into a task list. A single click on a button then validates the operations, and they are performed in one go.

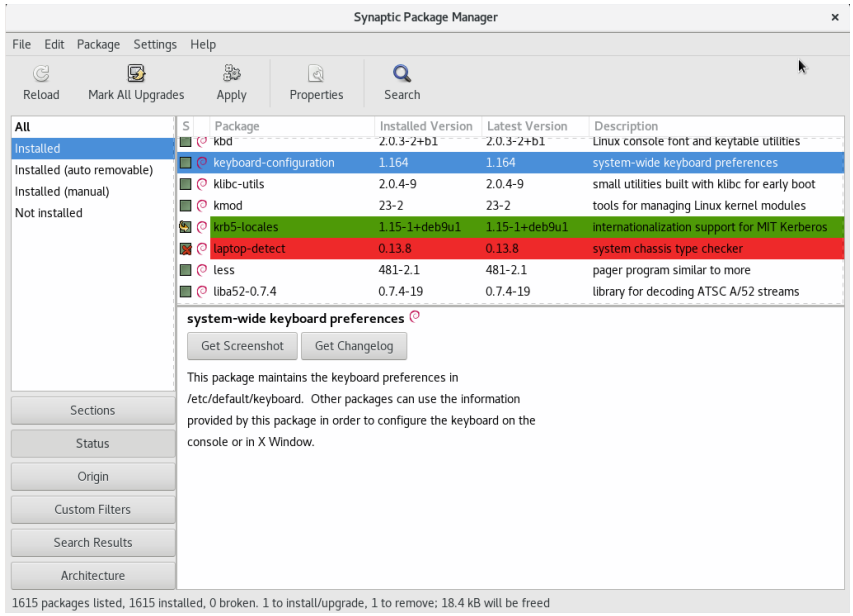


Figure 6.2 *synaptic package manager*

6.6. Checking Package Authenticity

Security is very important for Falcot Corp administrators. Accordingly, they need to ensure that they only install packages which are guaranteed to come from Debian with no tampering on the way. A computer cracker could try to add malicious code to an otherwise legitimate package. Such a package, if installed, could do anything the cracker designed it to do, including for instance disclosing passwords or confidential information. To circumvent this risk, Debian provides a tamper-proof seal to guarantee — at install time — that a package really comes from its official maintainer and hasn't been modified by a third party.

The seal works with a chain of cryptographical hashes and a signature and is explained in detail in `apt-secure(8)`. Starting with Debian 10 *Buster* the signed file is the `InRelease` file, provided by the Debian mirrors. There is also a legacy file called `Release`. Both contain a list of the `Packages` files (including their compressed forms, `Packages.gz` and `Packages.xz`, and the incremental versions), along with their SHA256 hashes, which ensures that the files haven't been tampered with. These `Packages` files contain a list of the Debian packages available on the mirror, along with their hashes, which ensures in turn that the contents of the packages themselves haven't been altered either. The difference between `InRelease` and `Release` is, that the former are cryptographically signed in-line, whereas the latter provide a detached signature in the form of the file `Release.gpg`.

NOTE

The future of `Release` and `Release.gpg`

Probably with the release of Debian 11 *Bullseye* APT will remove support for the legacy files `Release` and `Release.gpg`, used since APT 0.6, which introduced support for an archive authentication.

APT needs a set of trusted GnuPG public keys to verify signatures in the `InRelease` and `Release.gpg` files available on the mirrors. It gets them from files in `/etc/apt/trusted.gpg.d/` and from the `/etc/apt/trusted.gpg` keyring (managed by the `apt-key` command). The official Debian keys are provided and kept up-to-date by the *debian-archive-keyring* package which puts them in `/etc/apt/trusted.gpg.d/`. Note, however, that the first installation of this particular package requires caution: even if the package is signed like any other, the signature cannot be verified externally. Cautious administrators should therefore check the fingerprints of imported keys before trusting them to install new packages:

```
# apt-key fingerprint
/etc/apt/trusted.gpg.d/debian-archive-buster-automatic.gpg
-----------------------------------------------------------
pub   rsa4096 2019-04-14 [SC] [expires: 2027-04-12]
      80D1 5823 B7FD 1561 F9F7  BCDD DC30 D7C2 3CBB ABEE
uid           [ unknown] Debian Archive Automatic Signing Key (10/buster) <ftpmaster@debian.org>
sub   rsa4096 2019-04-14 [S] [expires: 2027-04-12]

/etc/apt/trusted.gpg.d/debian-archive-buster-security-automatic.gpg
-------------------------------------------------------------------
pub   rsa4096 2019-04-14 [SC] [expires: 2027-04-12]
      5E61 B217 265D A980 7A23  C5FF 4DFA B270 CAA9 6DFA
uid           [ unknown] Debian Security Archive Automatic Signing Key (10/buster) <ftpmaster@debian.org>
sub   rsa4096 2019-04-14 [S] [expires: 2027-04-12]

/etc/apt/trusted.gpg.d/debian-archive-buster-stable.gpg
-------------------------------------------------------
pub   rsa4096 2019-02-05 [SC] [expires: 2027-02-03]
      6D33 866E DD8F FA41 C014  3AED DCC9 EFBF 77E1 1517
uid           [ unknown] Debian Stable Release Key (10/buster) <debian-release@lists.debian.org>

/etc/apt/trusted.gpg.d/debian-archive-jessie-automatic.gpg
----------------------------------------------------------
pub   rsa4096 2014-11-21 [SC] [expires: 2022-11-19]
      126C 0D24 BD8A 2942 CC7D  F8AC 7638 D044 2B90 D010
uid           [ unknown] Debian Archive Automatic Signing Key (8/jessie) <ftpmaster@debian.org>

/etc/apt/trusted.gpg.d/debian-archive-jessie-security-automatic.gpg
-------------------------------------------------------------------
pub   rsa4096 2014-11-21 [SC] [expires: 2022-11-19]
      D211 6914 1CEC D440 F2EB  8DDA 9D6D 8F6B C857 C906
uid           [ unknown] Debian Security Archive Automatic Signing Key (8/jessie) <ftpmaster@debian.org>

/etc/apt/trusted.gpg.d/debian-archive-jessie-stable.gpg
-------------------------------------------------------
pub   rsa4096 2013-08-17 [SC] [expires: 2021-08-15]
      75DD C3C4 A499 F1A1 8CB5  F3C8 CBF8 D6FD 518E 17E1
uid           [ unknown] Jessie Stable Release Key <debian-release@lists.debian.org>

/etc/apt/trusted.gpg.d/debian-archive-stretch-automatic.gpg
-----------------------------------------------------------
pub   rsa4096 2017-05-22 [SC] [expires: 2025-05-20]
      E1CF 20DD FFE4 B89E 8026  58F1 E0B1 1894 F66A EC98
uid           [ unknown] Debian Archive Automatic Signing Key (9/stretch) <ftpmaster@debian.org>
sub   rsa4096 2017-05-22 [S] [expires: 2025-05-20]

/etc/apt/trusted.gpg.d/debian-archive-stretch-security-automatic.gpg
--------------------------------------------------------------------
pub   rsa4096 2017-05-22 [SC] [expires: 2025-05-20]
      6ED6 F5CB 5FA6 FB2F 460A  E88E EDA0 D238 8AE2 2BA9
uid           [ unknown] Debian Security Archive Automatic Signing Key (9/stretch) <ftpmaster@debian.org>
sub   rsa4096 2017-05-22 [S] [expires: 2025-05-20]

/etc/apt/trusted.gpg.d/debian-archive-stretch-stable.gpg
--------------------------------------------------------
pub   rsa4096 2017-05-20 [SC] [expires: 2025-05-18]
      067E 3C45 6BAE 240A CEE8  8F6F EF0F 382A 1A7B 6500
uid           [ unknown] Debian Stable Release Key (9/stretch) <debian-release@lists.debian.org>
```

IN PRACTICE

Adding trusted keys

When a third-party package source is added to the `sources.list` file, APT needs to be told to trust the corresponding GPG authentication key (otherwise it will keep complaining that it can't ensure the authenticity of the packages coming from that repository). The first step is of course to get the public key. More often than not, the key will be provided as a small text file, which we will call `key.asc` in the following examples.

To add the key to the trusted keyring, the administrator can just put it in a `*.asc` file in `/etc/apt/trusted.gpg.d/`. This is supported since Debian *Stretch*. With older releases, you had to run `apt-key add < key.asc`.

Once the appropriate keys are in the keyring, APT will check the signatures before any risky operation, so that front-ends will display a warning if asked to install a package whose authenticity can't be ascertained.

6.7. Upgrading from One Stable Distribution to the Next

One of the best-known features of Debian is its ability to upgrade an installed system from one stable release to the next: *dist-upgrade* — a well-known phrase — has largely contributed to the project's reputation. With a few precautions, upgrading a computer can take as little as a few minutes, or a few dozen minutes, depending on the download speed from the package repositories.

6.7.1. Recommended Procedure

Since Debian has quite some time to evolve in-between stable releases, you should read the release notes before upgrading.

BACK TO BASICS

Release notes

The release notes for an operating system (and, more generally, for any software) are a document giving an overview of the software, with some details concerning the particularities of one version. These documents are generally short compared to the complete documentation, and they usually list the features which have been introduced since the previous version. They also give details on upgrading procedures, warnings for users of previous versions, and sometimes errata.

Release notes are available online: the release notes for the current stable release have a dedicated URL, while older release notes can be found with their codenames:

➨ `https://www.debian.org/releases/stable/releasenotes`

➨ `https://www.debian.org/releases/stretch/releasenotes`

In this section, we will focus on upgrading a *Stretch* system to *Buster*. This is a major operation on a system; as such, it is never 100% risk-free, and should not be attempted before all important data has been backed up.

Another good habit which makes the upgrade easier (and shorter) is to tidy your installed packages and keep only the ones that are really needed. Helpful tools to do that include `aptitude`, `deborphan` and `debfoster` (see section 6.2.7, "Tracking Automatically Installed Packages" page 125). For example, you can use the following command, and then use `aptitude`'s interactive mode to double check and fine-tune the scheduled removals:

```
# deborphan | xargs aptitude --schedule-only remove
```

TIP

Finding changed files

The `debsums` command can check if files on the local system, which are part of an installed package, have been altered. It uses a simple hashsum algorithm and the information in `/var/lib/dpkg/info/`*`package`*`.md5sums` (see section 5.2.3, "Checksums, List of Configuration Files" page 89). To find all altered configuration files use `debsums -ec`. To check the whole system, use `debsums -c`.

Now for the upgrading itself. First, you need to change the `/etc/apt/sources.list` file to tell APT to get its packages from *Buster* instead of *Stretch*. If the file only contains references to *Stable* rather than explicit codenames, the change isn't even required, since *Stable* always refers to the latest released version of Debian. In both cases, the database of available packages must be refreshed (with the `apt update` command or the refresh button in `synaptic`).

NOTE

Repository information changes

When a new stable version of Debian is released, some fields in the `Release` and `InRelease` files of a repository change, like the `Suite` field. When this happens, downloading data from the repository is declined until the change is confirmed to ensure the user is prepared for it. To confirm the change use the `--allow-releaseinfo-change` or `--allow-releaseinfo-change-`*`field`* options for `apt-get` or the `Acquire::AllowReleaseInfoChange` configuration option.

Once these new package sources are registered, you should first do a minimal upgrade with `apt upgrade`. By doing the upgrade in two steps, we ease the job of the package management tools and often ensure that we have the latest versions of those, which might have accumulated bugfixes and improvements required to complete the full distribution upgrade.

Once this first upgrade is done, it is time to handle the upgrade itself, either with `apt full-upgrade`, `aptitude`, or `synaptic`. You should carefully check the suggested actions before applying them: you might want to add suggested packages or deselect packages which are only recommended and known not to be useful. In any case, the front-end should come up with a scenario ending in a coherent and up-to-date *Buster* system. Then, all you need is to do is wait while the required packages are downloaded, answer the debconf questions and possibly those about locally modified configuration files, and sit back while APT does its magic.

6.7.2. Handling Problems after an Upgrade

In spite of the Debian maintainers' best efforts, a major system upgrade isn't always as smooth as you could wish. New software versions may be incompatible with previous ones (for in-

stance, their default behavior or their data format may have changed). Also, some bugs may slip through the cracks despite the testing phase which always precedes a Debian release.

To anticipate some of these problems, you can install the *apt-listchanges* package, which displays information about possible problems at the beginning of a package upgrade. This information is compiled by the package maintainers and put in `/usr/share/doc/package/NEWS.Debian` files for the benefit of users. Reading these files (possibly through *apt-listchanges*) should help you avoid bad surprises.

You might sometimes find that the new version of a software doesn't work at all. This generally happens if the application isn't particularly popular and hasn't been tested enough; a last-minute update can also introduce regressions which are only found after the stable release. In both cases, the first thing to do is to have a look at the bug tracking system at https://bugs.debian.org/*package*[8], and check whether the problem has already been reported. If this is case it will be also listed before the upgrade begins, if you have *apt-listbugs* installed. If it hasn't, you should report it yourself with `reportbug`. If it is already known, the bug report and the associated messages are usually an excellent source of information related to the bug:

- sometimes a patch already exists, and it is available on the bug report; you can then recompile a fixed version of the broken package locally (see section 15.1, "Rebuilding a Package from its Sources" page 448);
- in other cases, users may have found a workaround for the problem and shared their insights about it in their replies to the report;
- in yet other cases, a fixed package may have already been prepared and made public by the maintainer.

Depending on the severity of the bug, a new version of the package may be prepared specifically for a new revision of the stable release. When this happens, the fixed package is made available in the proposed-updates section of the Debian mirrors (see section 6.1.2.3, "Proposed Updates" page 112). The corresponding entry can then be temporarily added to the `sources.list` file, and updated packages can be installed with `apt` or `aptitude`.

Sometimes the fixed package isn't available in this section yet because it is pending a validation by the Stable Release Managers. You can verify if that is the case on their web page. Packages listed there aren't available yet, but at least you know that the publication process is ongoing.

➨ `https://release.debian.org/proposed-updates/stable.html`

6.7.3. Cleaning Up after an Upgrade

APT usually ensures a clean upgrade, pulling in new and updated dependencies, or removing conflicting packages. But even being such a great tool, it cannot cover all tasks users and administrators will face after an upgrade, because they require a human decision.

[8] `https://bugs.debian.org`

Packages removed from the Debian Archive

Sometimes the Debian FTP Masters remove packages from the Debian archive, because they contain release critical bugs, were abandoned by their upstream author or their package maintainer, or simply reached their end of life. In this case a newer Debian release does not ship the package anymore. To find all packages, which do not have a package source, use the `apt-show-versions` command:

```
$ apt-show-versions | grep "No available version"
```

A similar result can be achieved by `aptitude search ~o`. If the packages found are not required anymore, they should be purged from the system, because they will not face any updates for critical or security related bugs anymore.

Dummy and Transitional Packages

Sometimes, it might be necessary for a package to get a new name. In this case often the old package is kept as an (almost) empty package, depending on the new one and installing only the mandatory files in `/usr/share/doc/`*`package`*`/`. Such packages are called "dummy" or "transitional" packages. If the package maintainer in charge also changed the section of this package to oldlibs, then tools like `aptitude`, `deboprhan`, or `debfoster` (see sidebar "`deborphan and debfoster`" page 125) can pickup these packages to suggest their removal.

Unfortunately there is currently no foolproof way of making sure that these packages are automatically removed or picked by the tools mentioned above. One way to check if the system still has some of these packages installed, is to look through the package descriptions of installed packages and then check the results. Be careful not to schedule the results for automatic removal, because this method can lead to false positives:

```
$ dpkg -l | grep ^ii | grep -i -E "(transition|dummy)"
```

Because the new package is pulled in as a dependency of the transitional package, it is usually marked as automatically installed and might be scheduled for removal if you try to purge the transitional package from your system. In this case you can use either of the approaches described in sidebar "Removing and installing at the same time" page 118 and section 6.2.7, "Tracking Automatically Installed Packages" page 125 to selectively remove the transitional package.

Old or Unused Configuration Files

If the upgrade was successful there might be some configuration file cruft, either from dpkg (see section 5.2.3, "Checksums, List of Configuration Files" page 89), ucf or from removed packages. The latter can be purged by using `apt autoremove --purge`. The configuration files, that were handled by *dpkg* or *ucf* during the upgrade process, have left some counterparts with a dedicated suffix (e.g. `.dpkg-dist`, `.dpkg-old`, `.ucf-old`). Using the `find` or `locate` command can help to track them down. If they are no longer of any use, they can be deleted.

Files not owned by any Package

The Debian policy enforces that packages don't leave files behind when they are purged. Violating this principle is a serious bug and you will rarely encounter it. If you do, report it; and if you are curious though, you can use the *cruft* or *cruft-ng* package to check your system for files not owned by any package.

6.8. Keeping a System Up to Date

The Debian distribution is dynamic and changes continually. Most of the changes are in the *Testing* and *Unstable* versions, but even *Stable* is updated from time to time, mostly for security-related fixes. Whatever version of Debian a system runs, it is generally a good idea to keep it up to date, so that you can get the benefit of recent evolution and bug fixes.

While it is of course possible to periodically run a tool to check for available updates and run the upgrades, such a repetitive task is tedious, especially when it needs to be performed on several machines. Fortunately, like many repetitive tasks, it can be partly automated, and a set of tools have already been developed to that effect.

The first of these tools is `apticron`, in the package of the same name. Its main effect is to run a script daily (via `cron`). The script updates the list of available packages, and, if some installed packages are not in the latest available version, it sends an email with a list of these packages along with the changes that have been made in the new versions. Obviously, this package mostly targets users of Debian *Stable*, since the daily emails would be very long for the faster paced versions of Debian. When updates are available, `apticron` automatically downloads them. It does not install them — the administrator will still do it — but having the packages already downloaded and available locally (in APT's cache) makes the job faster.

Administrators in charge of several computers will no doubt appreciate being informed of pending upgrades, but the upgrades themselves are still as tedious as they used to be. Periodic upgrades can be enabled: it uses a `systemd` timer unit or `cron`. If *systemd* is not installed, the `/etc/cron.daily/apt-compat` script (in the *apt* package) comes in handy. This script is run daily (and non-interactively) by `cron`. To control the behavior, use APT configuration variables (which are therefore stored in a file `/etc/apt/apt.conf.d/10periodic`). The main variables are:

APT::Periodic::Update-Package-Lists This option allows you to specify the frequency (in days) at which the package lists are refreshed. `apticron` users can do without this variable, since `apticron` already does this task.

APT::Periodic::Download-Upgradeable-Packages Again, this option indicates a frequency (in days), this time for the downloading of the actual packages. Again, `apticron` users won't need it.

APT::Periodic::AutocleanInterval This option covers a feature that `apticron` doesn't have. It controls how often obsolete packages (those not referenced by any distribution

anymore) are removed from the APT cache. This keeps the APT cache at a reasonable size and means that you don't need to worry about that task.

APT::Periodic::Unattended-Upgrade When this option is enabled, the daily script will execute `unattended-upgrade` (from the *unattended-upgrades* package) which — as its name suggest — can automatize the upgrade process for some packages (by default it only takes care of security updates, but this can be customized in `/etc/apt/apt.conf.d/50unattended-upgrades`). Note that this option can be set with the help of debconf by running `dpkg-reconfigure -plow unattended-upgrades`. If *apt-listbugs* is installed it will prevent an automatic upgrade of packages which are affected by an already reported serious or grave bug.

Other options can allow you to control the cache cleaning behavior with more precision. They are not listed here, but they are described in the `/usr/lib/apt/apt.systemd.daily` script.

These tools work very well for servers, but desktop users generally prefer a more interactive system. The package *gnome-software* provides an icon in the notification area of desktop environments when updates are available; clicking on this icon then runs an interface to perform updates. You can browse through available updates, read the short description of the relevant packages and the corresponding `changelog` entries, and select whether to apply the update or not on a case-by-case basis.